Edge deployment: Plant health classification (MobileNet V3) on Raspberry Pi Zero 2 W

Yusuf Savaş

Advisor: Fikret Gürgen

June 1, 2026

TABLE OF CONTENTS

1. Introduction

Running machine learning on the edge means running models on small devices near where data is collected or where people use them, not only in big cloud servers. That helps when you care about fast responses, keeping data local, unreliable internet, or saving battery and power. A typical approach is to save a trained model in a portable format and run it with a small runtime on a compact computer or even a tiny microcontroller.

This work focuses on **edge deployment of a plant health classifier**: given a leaf image, the system predicts whether the plant appears **healthy** or **diseased**. The **motivation** includes:

- **Engineering**: validate that a compact CNN (**MobileNet V3** [1]) can run end-to-end on constrained hardware (**Raspberry Pi Zero 2 W** [2]) with acceptable accuracy and measurable throughput.
- **Learning**: understand the **role of quantization and INT8-oriented optimization** [3] in that setting. Quantization reduces numeric precision (often from FP32 to INT8) to shrink model size and speed up CPU inference; the trade-off is potential accuracy loss. Documented results establish an **FP32 ONNX baseline** on the Pi so a later **INT8** deployment can be compared on accuracy, latency, and throughput.
- **Native code vs. Python on the edge**: **C++** skips the Python interpreter and **GIL**, so it can run faster on a small CPU and use **512 MB RAM** more predictably. **ONNX Runtime** [4] is often called from **native** code in production. **Python** is still a good fit for training and quick tests; **C++** is a common choice for **always-on** or **camera-stream** inference on tight hardware.
- **Real-world validation**: Even if a model does well on well-chosen leaf photos, we need to see how it works "in real life" with real leaves, real lighting, and photos taken with an actual camera. This testing checks if the model handles blur, bad

lighting, background mess, and camera noise. It helps find problems before the system is used for real.

2. Description

2.1. Hardware

Raspberry Pi Zero 2 W was chosen as a **low-cost, resource-constrained** edge node:

Item	**Detail**
CPU	Quad-core ARM Cortex-A53 (∼1 GHz class)
RAM	512 MB
Acceleration	inference is **CPU-bound**
Role	Baseline for always-on or field-style deployments

2.2. Software

- **Model**: **MobileNet V3** [1] exported to **ONNX** [5] for deployment.
- **Runtime**: **ONNX Runtime** [4] used from **C++**, not Python, so latency and FPS reflect the actual on-device stack instead of extra interpreter overhead.
- **Pipeline**: **Image in** → **MobileNet-style preprocessing** → **ONNX Runtime inference** → **class scores / label**.
- **Web UI**: a **web portal** on the device (or on the LAN) shows **camera view**, **live predictions**, **summary metrics**, and **throughput (e.g. FPS or images per second)**.

2.3. Use

In practice the app does one thing: from a picture of a leaf it guesses whether the plant looks **healthy** or **unhealthy**.

For a **demo**, you open the **web page** on your phone or laptop (on the same

Wi-Fi as the device). You show **healthy** and **unhealthy** plants to the camera, or step through example images, and everyone can see the **label** the model picked and **how many frames per second** it is running.

2.4. Diagram

Figure 2.1 shows the logical data flow.

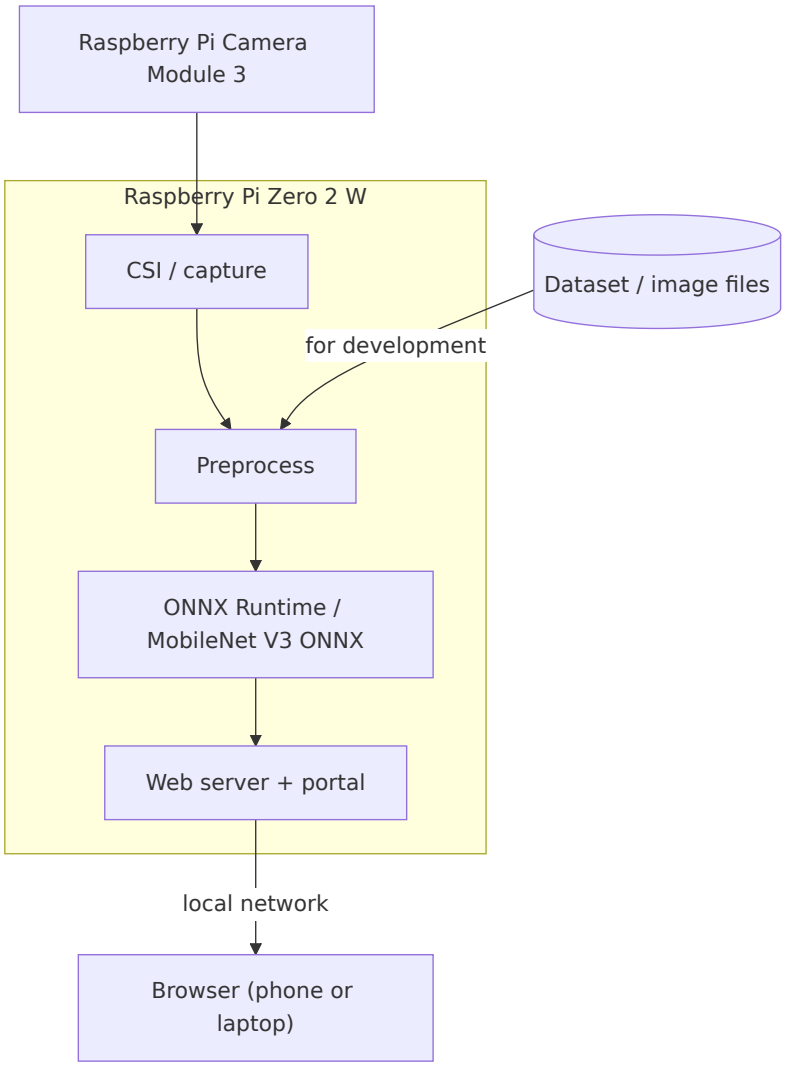

Figure 2.1. System architecture: camera and files into preprocess, ONNX Runtime, web portal, and browser on the LAN.

3. Requirements

3.1. Features

- Open a **web portal** and see **inference results** (predicted class and, where applicable, confidence).
- See **performance at a glance**: **FPS** (or equivalent throughput) and, if useful, **per-frame or rolling latency**, updated as the system runs.
- Run **single-image** or **stream/batch** inference through the same stack so lab tests and field-style runs share one pipeline.

Layers

Layer	**Role**
Presentation	Web UI (portal) for status, predictions, FPS, and optional history or session summary.
Application / API	Serves the UI and exposes inference status and metrics to it (e.g. HTTP or WebSocket).
Inference	Loads the **MobileNet V3 ONNX** graph, applies consistent preprocessing, runs **ONNX Runtime** on the device CPU.
Acquisition (optional path)	Feeds frames from storage or from a **camera** into the preprocessor; same graph for file replay and live capture when implemented.
Quantization track	Same topology with **INT8** weights for comparison against the **FP32** baseline, with accuracy and speed measured under identical UI and metrics.

Offline tests

- Ability to score a **fixed labeled test split** and report **classification metrics** (accuracy, balanced accuracy, precision/recall/F1 for the diseased class, specificity) and a **confusion matrix**, independent of the portal, for rigorous comparison between FP32 and INT8 builds.

3.2. Quality

- **Deployability**: run on **512 MB RAM** and a **quad-core A53** CPU without a GPU on the edge device.
- **Reproducibility**: evaluation procedures and hardware context are recorded so accuracy and throughput numbers remain comparable across model variants.
- **Observability**: **accuracy** (from labeled runs) is separable from **latency / throughput** (portal and benchmarks); end-to-end timing includes load, preprocess, and runtime where applicable.

4. Designed System

4.1. Choices

- **MobileNet V3** [1] balances accuracy and a small parameter count (**1,519,906** parameters in the trained checkpoint) for edge use.
- **ONNX + ONNX Runtime** [4,5] provides a **deployment-oriented** graph and a runtime tuned for multiple backends; a **native** caller keeps measurements close to how the stack would ship in production.
- **FP32 baseline first**, then **INT8** (or similar) as a controlled experiment: smaller artifacts and faster integer math on CPU, with accuracy tracked against the same test protocol.
- **Web portal** as the default way to **see results and FPS** aligns the system with operator expectations and simplifies demos and field checks.

4.2. Pipeline

(i) **Input**: image from file, batch folder, or (when available) camera frame.
(ii) **Preprocess**: resize and normalize to match training/export conventions.
(iii) **Inference**: ONNX Runtime executes the graph on the CPU.
(iv) **Output**: class prediction (and optional scores); metrics feed the **portal** and any batch evaluator.
(v) **Presentation**: portal displays **prediction**, **FPS / throughput**, and optional aggregates.

4.3. Quantization

A reduced-precision build (e.g. **INT8** weight quantization) targets **smaller footprint and faster inference** on the same CPU, with **accuracy and FPS** compared to the **FP32** baseline under the same preprocessing and portal-facing metrics.

5. Implementation

The system is implemented in two stages: a **Python** training/export pipeline on a development workstation, and a **C++** on-device runtime that serves a web portal on the Raspberry Pi. Both stages share the same tensor contract (input `[1,3,224,224]` float32 NCHW, ImageNet normalization) so on-device results are comparable to the reference evaluation.

5.1. Data preparation

Two scripts build a three-class dataset (**healthy**, **diseased**, **background**):

- `prepare_data.py` downloads **PlantVillage** [6] through TensorFlow Datasets. The 38 fine-grained PlantVillage labels are collapsed into two classes: any label containing `healthy` maps to **healthy**, all others to **diseased**.
- `prepare_background_data.py` adds a **background** class from **COCO 2017** [7] (plus a small synthetic fraction) so the deployed model can reject non-leaf scenes from a live camera.

Images are split **70 / 15 / 15** into train/val/test and written to `data/{train,val,test}/{healthy,diseased,background}/`.

5.2. Training pipeline (Python)

- **Model** (`models/mobilenet_v3.py`): wraps torchvision `mobilenet_v3_small` with ImageNet-pretrained weights; the classifier head is replaced with `Dropout(0.2)` followed by a `Linear` layer sized to the number of classes.
- **Data loading** (`utils/data_loader.py`): resize to **224×224**, ImageNet normalization (mean `[0.485,0.456,0.406]`, std `[0.229,0.224,0.225]`); training augmentations are horizontal flip, ±15° rotation, and color jitter; a `WeightedRandomSampler`

balances the classes.

- **Training** (`train.py`, config in `models/registry.py`): **15** epochs, batch size **32**, **Adam** (lr `1e-4`, weight decay `1e-4`), `CrossEntropyLoss`; the best checkpoint (lowest validation loss) is saved to `checkpoints/mobilenet_v3_3cls_best.pth` with class metadata.
- **Evaluation** (`evaluate.py`, `utils/evaluation.py`): scikit-learn metrics and confusion matrix on the held-out test split.

5.3. ONNX export and quantization

`export_mobilenet_onnx.py` exports the checkpoint to `checkpoints/mobilenet_v3_3cls.onnx` with input name `input`, output name `logits`, shape `[1,3,224,224]`, and **opset 17**. Class names and counts are embedded in the ONNX `metadata_props`, and an optional PyTorch-vs-ONNX-Runtime parity check asserts a maximum absolute difference below `1e-3`. A reduced-precision **INT8/QDQ** build is produced in ONNX Runtime's `.ort` format (`mobilenet_v3_3cls_qdq.ort`) for the quantization comparison.

5.4. On-device runtime (C++)

The C++ code lives under `cpp/` as a **CMake** project (C++17, ONNX Runtime **1.24.4**) with the following components:

- **Preprocessing** (`cpp/src/preprocess/mobilenet_preprocess.cpp`): a single fused pass performs bilinear resize, optional R/B channel swap, and ImageNet normalization into the NCHW tensor; a **NEON**-accelerated path is used on aarch64.
- **Inference** (`cpp/src/inference_ort/ort_engine.cpp`): loads the ONNX/ORT graph, runs an ONNX Runtime session via IO binding, and applies softmax and argmax to produce a label and probabilities.
- **Camera** (`cpp/src/camera_libcamera/libcamera_camera.cpp`): captures frames

through **libcamera** [8] and converts them to RGB888.

- **HTTP server** (`cpp/src/server_http/http_stream_server.cpp`, using **cpp-httplib**): serves the UI at `/`, an MJPEG preview at `/stream.mjpg`, inference events via Server-Sent Events at `/events`, and device telemetry at `/metrics`.
- **Live application** (`cpp/apps/live_infer_web/main.cpp`): wires camera → preprocess → engine → web display together; default port **8080**.
- **Batch tool** (`cpp/tools/evaluate_mobilenet.cpp`, `phc_evaluate_mobilenet`): scores a labeled folder and reports accuracy, per-class metrics, a confusion matrix, and throughput. This tool produced the on-device numbers in the next chapter.

5.5. Web portal

The portal is a single self-contained page (`web/live/index.html`, vanilla HTML/CSS/JavaS
embedded into the binary at build time via `cpp/cmake/embed_html.cmake`. It shows the MJPEG preview (`<img src="/stream.mjpg">`), subscribes to predictions with `EventSource('/events')`, renders per-class probability bars, computes **FPS** from a rolling one-second window of events, and polls `/metrics` once per second for CPU load, memory, and temperature.

5.6. Testing, cross-compilation, and deployment

Unit tests use **Catch2** (`cpp/tests/test_preprocess.cpp`) to validate tensor shape and channel handling, and `scripts/validate_cpp_inference.sh` checks Python-vs-C++ parity on identical preprocessed tensors. The Pi target is built with the cross toolchain in `cpp/toolchains/rpi-aarch64/`; `scripts/download_onnxruntime.sh` fetches the matching ONNX Runtime, and `scripts/deploy_rpi_zero2w.sh` pushes the binary and model to the device.

6. Demonstration

This chapter first shows the system in everyday use through the **live web portal**, then reports **benchmark-style** runs on the edge device (Raspberry Pi Zero 2 W, ONNX Runtime via native code) and a **development workstation** reference (PyTorch on **CUDA**). On-device runs cover both **FP32** ONNX and **INT8/QDQ** ORT builds; sample counts are noted per table (**8106** images for the 2-class leaf-only split, **12606** images for the 3-class split that includes the background class).

6.1. Live web portal

In a demo, the device runs `live_infer_web` and the operator opens `http://<pi-ip>:8080/` from a phone or laptop on the same network. The page (Figure 2.1 shows the underlying data flow) presents the **live camera preview**, the **predicted label** with per-class **probability bars**, the current **throughput (FPS)**, and **device metrics** (CPU load, memory, temperature). Showing healthy and diseased leaves to the camera, or stepping through example images, updates the label and probabilities in real time while the background class lets the model reject non-leaf scenes.

6.2. Pi Zero 2 W – FP32 (2-class, 8106 images)

These numbers use **FP32** ONNX on the leaf-only test split (**INT8** not used here).

Metrics (8106 test images)

Confusion matrix

TN: 2215, FP: 7, FN: 7, TP: 5877

Metric	Value
Accuracy	0.9983 (99.83%)
Balanced accuracy	0.9978 (99.78%)
Precision (diseased)	0.9988 (99.88%)
Recall (diseased)	0.9988 (99.88%)
F1 (diseased)	0.9988 (99.88%)
Specificity (healthy)	0.9968 (99.69%)

	Pred: healthy	Pred: diseased
Actual: healthy	2215	7
Actual: diseased	7	5877

Speed (load, preprocess, infer; 8106 images)

Measure	Value
Total time	1009.689 s
Avg / image	124.561 ms
Throughput	8.028 img/s

6.3. Pi Zero 2 W – FP32 (3-class, 12606 images)

The full three-class split adds the **background** class. This run uses **4 threads**.

Overall metrics

Confusion matrix

Per-class metrics

Speed (load, preprocess, infer; 12606 images, 4 threads)

Metric	Value
Accuracy	0.9981 (99.81%)
Balanced accuracy	0.9979 (99.79%)
Macro precision	0.9971
Macro recall	0.9979
Macro F1	0.9975

	Pred: healthy	Pred: diseased	Pred: background
Actual: healthy	2215	5	2
Actual: diseased	13	5871	0
Actual: background	3	1	4496

Class	Precision	Recall	F1	Support
Healthy	0.9928	0.9968	0.9948	2222
Diseased	0.9990	0.9978	0.9984	5884
Background	0.9996	0.9991	0.9993	4500

Measure	Value
Total time	801.375 s
Avg / image	63.571 ms
Throughput	15.730 img/s

6.4. Pi Zero 2 W – INT8/QDQ (3-class, 12606 images)

The INT8/QDQ ORT build (`mobilenet_v3_3cls_qdq.ort`) trades a few accuracy points for a smaller footprint. Two runs are reported: a single-thread run and a 4-thread run.

Overall metrics

Metric	INT8 (1 thread)	INT8 (4 threads)
Accuracy	0.9721 (97.21%)	0.9705 (97.05%)
Balanced accuracy	0.9691 (96.91%)	0.9667 (96.67%)
Macro precision	0.9605	0.9588
Macro recall	0.9691	0.9667
Macro F1	0.9646	0.9626

Confusion matrix (INT8, 1 thread)

	Pred: healthy	Pred: diseased	Pred: background
Actual: healthy	2106	108	8
Actual: diseased	218	5658	8
Actual: background	3	7	4490

Per-class metrics (INT8, 1 thread)

Class	Precision	Recall	F1	Support
Healthy	0.9050	0.9478	0.9259	2222
Diseased	0.9801	0.9616	0.9707	5884
Background	0.9964	0.9978	0.9971	4500

Speed (load, preprocess, infer; 12606 images)

Measure	INT8 (1 thread)	INT8 (4 threads)
Total time	2085.312 s	1282.092 s
Avg / image	165.422 ms	101.705 ms
Throughput	6.045 img/s	9.832 img/s

Build (split, threads)	Accuracy	Throughput
FP32 (2-class, 8106)	99.83%	8.028 img/s
FP32 (3-class, 12606, 4 thr.)	99.81%	15.730 img/s
INT8/QDQ (3-class, 12606, 1 thr.)	97.21%	6.045 img/s
INT8/QDQ (3-class, 12606, 4 thr.)	97.05%	9.832 img/s

6.5. Summary comparison

On the Pi Zero 2 W, **FP32** keeps accuracy near the reference while **multi-threading** roughly doubles throughput. The **INT8/QDQ** build loses about **2.5–2.7** accuracy points (mostly on the healthy/diseased boundary) and, in this ORT configuration, does not beat multi-threaded FP32 on speed, so FP32 with 4 threads is the strongest on-device operating point measured here.

6.6. Dev PC (GPU, reference)

Setup (example): Ubuntu 24.04 LTS on **WSL2**; **CPU**: AMD Ryzen 7 5800H; **GPU**: NVIDIA GeForce RTX 3050 Laptop (4 GB VRAM).

Test metrics

Confusion matrix

TN: 2212, FP: 10, FN: 6, TP: 5878

Speed (full eval loop)

Metric	Value
Accuracy	0.9980 (99.80%)
Balanced accuracy	0.9972 (99.72%)
Precision (diseased)	0.9983 (99.83%)
Recall (diseased)	0.9990 (99.90%)
F1 (diseased)	0.9986 (99.86%)
Specificity (healthy)	0.9955 (99.55%)
MCC	0.9950
ROC-AUC	1.0000
Parameters	1,519,906

	Pred: healthy	Pred: diseased
Actual: healthy	2212	10
Actual: diseased	6	5878

Measure	Value
Total	10.7903 s
Avg / image	1.331 ms
Throughput	751.23 img/s

7. Conclusions and Future Work

On the **Pi Zero 2 W**, **FP32 ONNX** with **ONNX Runtime** stays accurate on the held-out test set (∼99.8%) and runs at about **8 images/s** single-threaded, rising to about **16 images/s** with four threads on the three-class split. The **INT8/QDQ** build trades roughly **2.5–2.7** accuracy points for a smaller artifact but, in this ORT configuration, did not outperform multi-threaded FP32 on throughput; multi-threaded FP32 is therefore the strongest on-device operating point measured here. These results give a solid baseline before further speedups. **Testing with a real camera in real conditions** still matters, because lab-style test images do not cover everything you see in the field.

7.1. Future work

- **Field tests**: systematic runs **outdoors / in greenhouses** with **natural lighting** and varied distances, not only dataset-style images, now that the live camera path exists.
- **Quantization recovery**: close the INT8 accuracy gap with **quantization-aware training** or per-channel calibration, and tune runtime flags and threading for Cortex-A53 to raise images per second.
- **UI**: make **FPS**, latency, and errors easy to see when inference or the camera degrades.
- **Calibration**: optional score calibration or thresholds for diseased vs healthy when moving from lab to field.

REFERENCES

1. Howard, A., M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le and H. Adam, "Searching for MobileNetV3", *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.

2. Raspberry Pi Ltd., "Raspberry Pi Zero 2 W", `https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/`, 2021, accessed 2026.

3. Jacob, B., S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference", *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

4. ONNX Runtime developers, "ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator", `https://onnxruntime.ai`, 2021, accessed 2026.

5. ONNX Community, "ONNX: Open Neural Network Exchange", `https://onnx.ai`, 2019, accessed 2026.

6. Hughes, D. P. and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics", *arXiv preprint arXiv:1511.08060*, 2015.

7. Lin, T.-Y., M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick, "Microsoft COCO: Common Objects in Context", *European Conference on Computer Vision (ECCV)*, pp. 740–755, Springer, 2014.

8. libcamera developers, "libcamera: An open source camera stack and framework for Linux, Android, and ChromeOS", `https://libcamera.org`, 2024, accessed 2026.